
Running the studio from one screen

Everything `/studio` does, in the order you actually do it. Written against what is deployed, not what is planned. **Where this file and the app disagree, the app is right and this file is stale** — fix it in the same commit that changed the UI.

Getting in

Sign in at **brandmintstudios.in/login** as `admin@brandmintstudios.in`. You land on `/studio`. Anyone who is not that address lands on `/portal` instead, and there is no route from there to the studio screen.

Two shortcuts worth learning on day one:

- `/` or `⌘K` — search every client, project, lead and invoice by name.
- **New** — creates whatever the section you are in creates.

The sixty seconds that matter

Today is the only screen you have to open. Four numbers and a list, all computed from your own data.

This month is signed retainers *plus* what live builds earn per month, shown split and never merged. They behave differently: retainer money comes back next month, build money stops the day it launches.

Two builds running does not double it. Every live build is pooled — total agreed value over total agreed days, times a month of working days — so taking on a third **lengthens the schedule** instead of inflating the month. See `monthlyIncome()` in `bm-app.js` and `tests/income.test.mjs`, which asserts exactly that.

The action list is ordered by what costs most to ignore: an unanswered lead, then clients sitting on your requests, then work awaiting approval, then money. When it is genuinely clear it says so; when the system is *empty* it says that instead, because an empty database and a finished day look identical on most dashboards and mean opposite things.

A proposal is never revenue. A retainer agreed but not marked signed counts as zero everywhere. Today tells you separately how much is sitting unsigned, so the gap is visible without being counted.

A. Taking on a client

One screen, four answers. It replaced seven steps across three pages, which is why it kept getting abandoned halfway.

1. **Clients → New engagement** (the same button is on Delivery).
2. **Name them, then pick the deal** — One-time project · Project then retainer · Retainer only. The form asks only for what that shape needs.
3. **Price and weeks.** One number each. Optionally pick the kind of work, which stamps a milestone schedule for free.
4. **Read the preview.** It writes out exactly what is about to be created — the build, both invoice dates and amounts, and the monthly run rate while it runs. If that sentence is wrong, fix it before submitting rather than after.
5. **Create.** One submit makes the client, the project, the agreed scope line, the 50% now / 50% at launch invoices, and the schedule.

The scope becomes **one** line, not an invented four-phase breakdown — a breakdown nobody agreed to is fiction with a progress bar attached. Refine it on `/quote` when a real one exists.

No path through this form marks a retainer signed unless you tick the box.

B. Giving them a login

The one job that happens in two places, and the one most likely to be left half-done.

1. **Firebase Console → Authentication → Users → Add user.** Email is `theirname@brandmintstudios.in`. **Set the password there.** Copy the UID.
2. **Studio → Access.** Username, display name, organisation, paste the UID. The form shows the exact email they will sign in with before you save.
3. **Check it says "Saved."** They appear under *Existing logins* marked `linked`.

Do both halves or neither works. The Firebase account and the `users` document are separate writes. A client with the account but no `users` document signs in successfully and then **reads nothing at all** — not even their own company name, because the rules resolve every read through that document. A blank portal almost always means step 2 was skipped.

Never collect a client credential. No screen here accepts one and you should not take one by email either. When a client needs to give you access to something of theirs, ask them to add `hello@brandmintstudios.in` as a user on their own account.

The three ways you sell

DEAL	BUILD	RETAINER	WHAT DELIVERY SHOWS
One-time	yes, 50/50	none, by agreement	% delivered, from the scope
Then retainer	yes, 50/50	starts at launch	% delivered, then ongoing
Retainer only	no	the whole engagement	Ongoing — no end date, by design

A retainer engagement is never nagged about missing milestones. It does not have any and is not supposed to.

What each section is for

SECTION	FOR
Today	The month, and what needs you. Open this one.
Pipeline	Leads as a board. Log first response the moment you reply — stamped once, never moves, and the most valuable number in the business.
Clients	One row each → retainer, projects, invoices, login, and which onboarding step is next. Mark a retainer signed here.
Delivery	One row per project → move scope lines <code>agreed → building → delivered → accepted</code> with a click, raise what you need from the client, put work up for approval.
Money	Every invoice ever raised. <i>Record payment</i> takes the total received , so a part payment is a real number.
Access	Client logins, and who has one.
Activity	Everything that moved money, access or delivery status. Append-only — nobody can edit or delete an entry, including you .

The daily habit

Three things, none longer than a minute, and between them every number on the screen stays true.

Raise what you need from the client. Delivery → open the project → add each item under *What we need from the client*. This is the whole reason the portal exists: it shows them, dated, so you are not the one chasing. A project with nothing raised has that switched off, and Today will tell you so.

Move a scope line when it moves. One click. The percentage, the client's portal and the month all follow without anything being typed twice. A line counts as money only at **accepted**

— delivered means handed over, not confirmed.

Record money the day it lands. Money → *Record payment* → the total received. For anything paid online use **Sync payments** instead: it asks Razorpay what it was actually paid rather than taking your word for it.

Still outstanding

- **Revoke the old service-account key** `090ec957...`. It was pasted into a chat once, which makes it spent whatever happened to the file. Firebase Console → Project settings → Service accounts. Nothing in the app needs it.
- **Publish** `firestore.rules`. Firestore → Rules → paste → Publish. Until then the Activity log records nothing and says so plainly.
- **Prove tenancy on live data.** Sign in as a real client and open `/tenancy-check`. It should say **Isolated**. It is tested six ways against the emulator and has never been run against the live database — §9 item 1.

Brand Mint Studios · generated from `docs/WALKTHROUGH.md`. Regenerate with `node tools/make-manual.mjs` whenever the admin screen changes.